

AaltoFlow

Python software for instrument control and multi-dimensional scans.

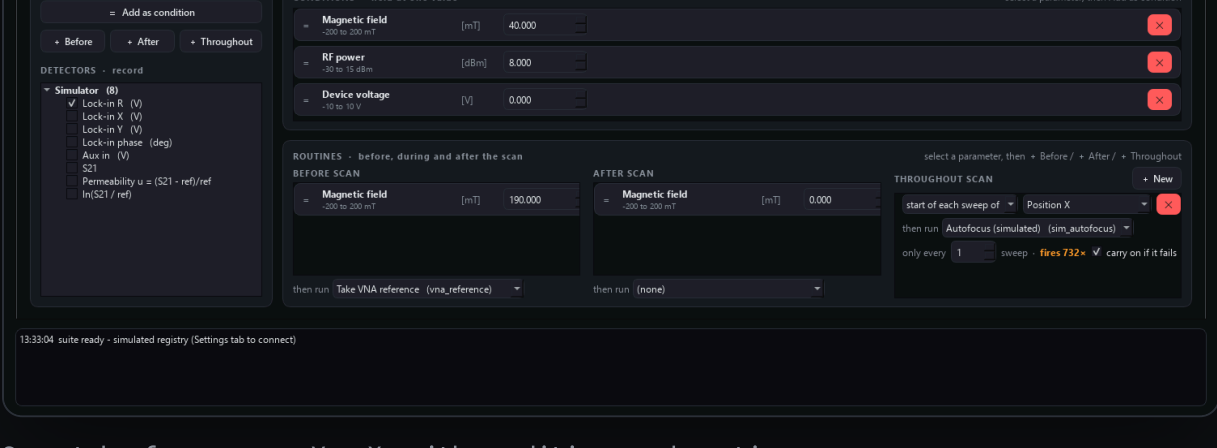
Each instrument runs as its own small program (a service) with its own window, and the programs communicate over the network. A scan engine drives any combination of them and writes netCDF files. It was written to replace the LabVIEW control software of the time-resolved MOKE setup in the NanoSpin group. The camera, the KIM stage and the power meter have been used on that setup; the other instruments have so far run in simulation.

- Source on GitHub
- AaltoView (data viewer)
- DOI

12 instrument modules · any number of scan axes · a simulator in each module · MIT licence · netCDF data including the scan settings

Scan definition

Parameters of the connected instruments are arranged as nested axes, outer to inner; the number of axes is not limited. Other parameters can be held at fixed values, which are saved with the data. Routines run before, during or after the scan, for example a VNA reference at a reference field, or an autofocus at the start of each row.



Scan tab: frequency × Y × X, with conditions and routines



Measurement tab: 41 × 61 field–frequency map (simulation)

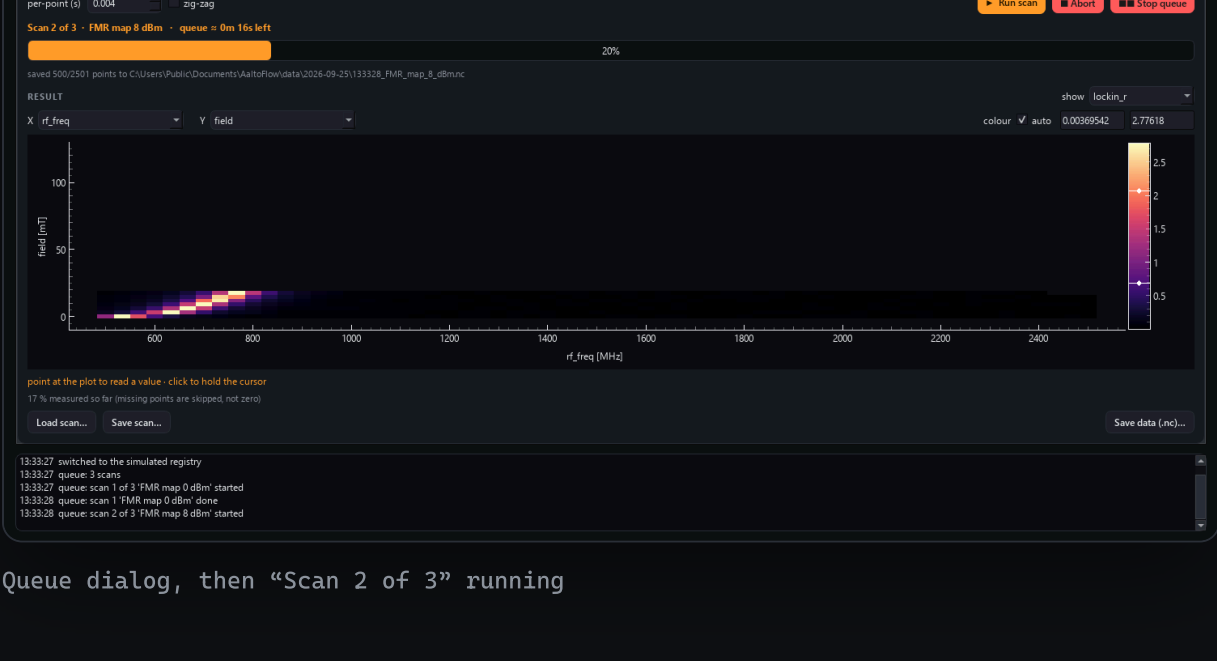
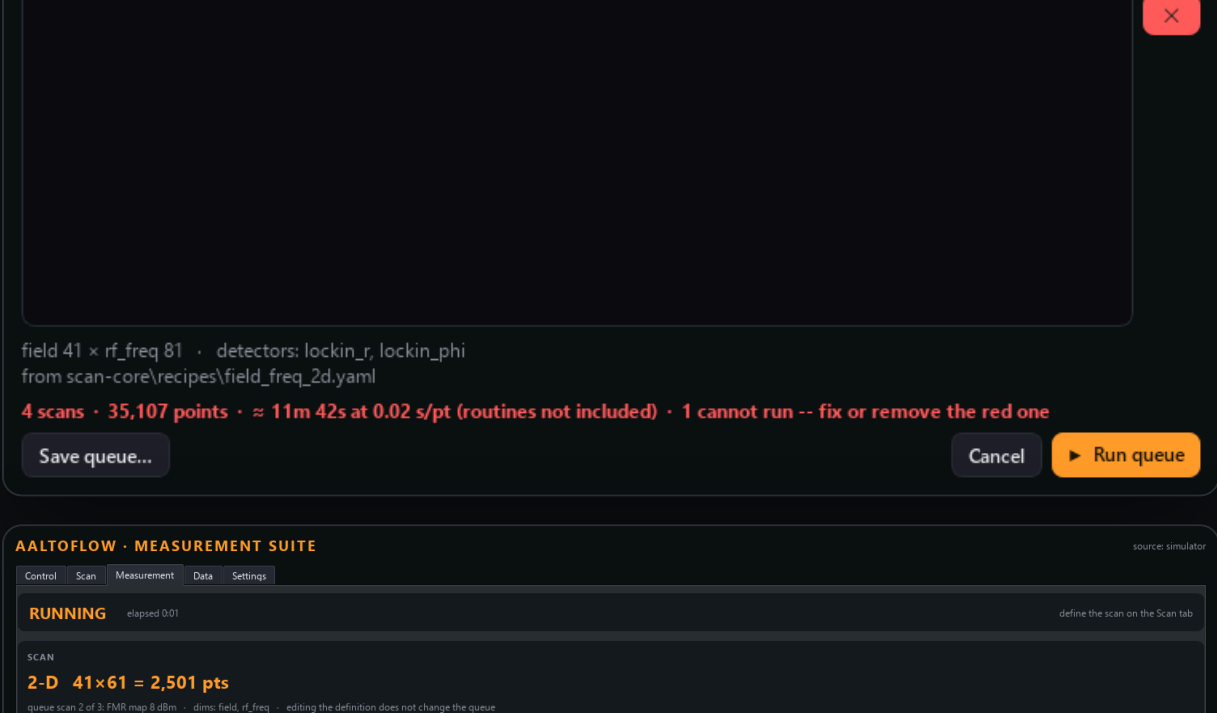
Live data and saving

The plot is updated during the scan; the example is a simulated field against frequency FMR map. Values can be read with a cursor. Every run is saved automatically, and scans with more than 100 points are also saved after every tenth of the points.

QUEUE

Queue of scans

Several scan definitions can be loaded, named and ordered. All of them are checked against the connected instruments before the first one starts. Abort ends the current scan and the queue continues with the next; each scan is saved to its own file.



AaltoView: pos_x × pos_y at 45 mT, one resonant element

ANALYSE

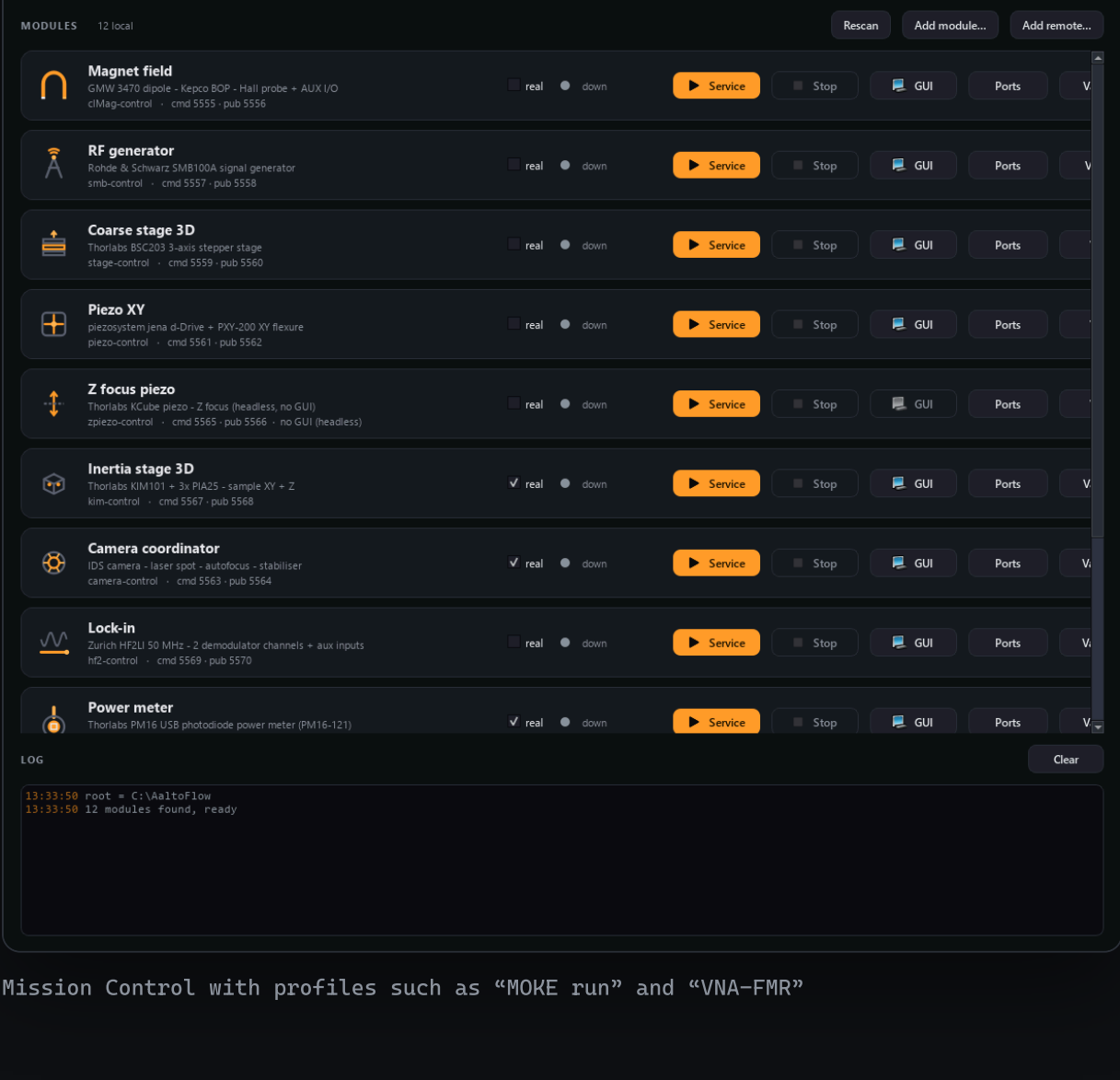
Data viewer (AaltoView)

Any two dimensions of a data set are shown as a map; each remaining dimension is held at one value or averaged. The example is a simulated FMR image of a patterned sample at 45 mT. Export to Origin, to a Jupyter notebook that recomputes the view, to image files or to CSV.

LAUNCH

Mission Control

The launcher lists the installed instrument modules, starts their services in the required order and opens their windows. A profile starting a predefined set of modules. Instruments running on another PC of the lab network can be added.

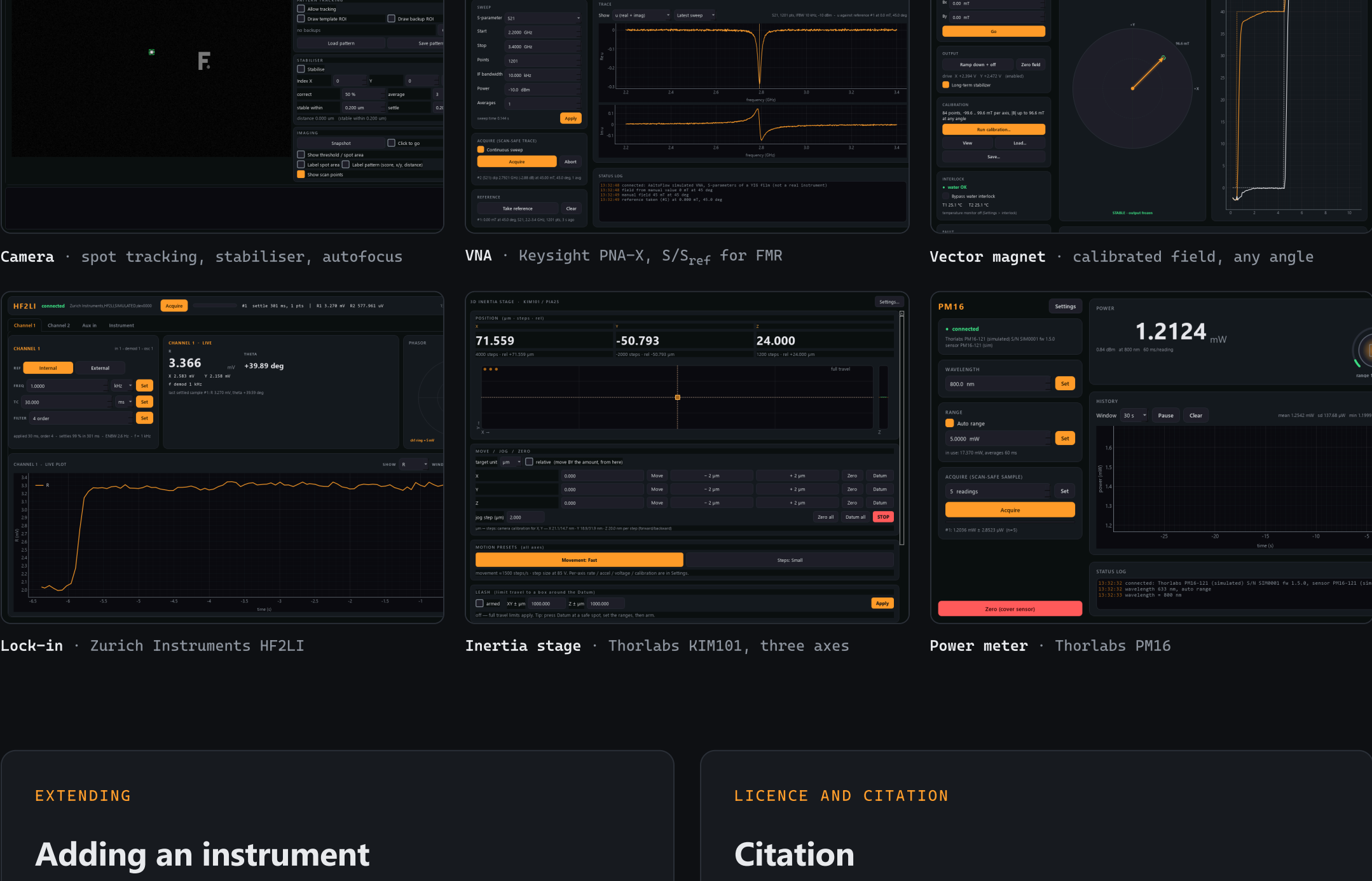


Mission Control with profiles such as “MOKE run” and “VNA–FMR”

INSTRUMENTS

Instrument modules

Electromagnets, an RF generator, stages and piezos, a microscope camera, a lock-in amplifier, a power meter and a VNA. Each module includes a simulator, so the software can be tried without the hardware.



EXTENDING

Adding an instrument

A new module is generated from an existing one and given the instrument's driver. Each module reports its own parameters, limits and units, so the launcher, the control panel and the scan builder list it without further changes.

```
# a new module from a template, next free port pair
python tools/new_module.py laser --like smb
cd laser-control
uv sync --extra gui; uv run pytest -q
# check it speaks the protocol
python tools/check_modules.py laser --live
```

LICENCE AND CITATION

Citation

AaltoFlow and AaltoView are released under the MIT licence. If you publish data measured or analysed with them, please cite:

AaltoFlow · doi:10.5281/zenodo.22959230

AaltoView · doi:10.5281/zenodo.22959226

L. Flajsman, *AaltoFlow: lab automation built from independent instrument modules*, NanoSpin group, Aalto University, doi:10.5281/zenodo.22959230

Copy citation